

Final Report: Project Semester
on
Development of Smart-Cart Application Using Java Spring Boot

Submitted by
Yash Bansal
102219073
(B.E., Electrical & Computer Engineering)

Under the Guidance of

Host Mentor
Confidential
Technology & Transformation Practice
Deloitte Touche Tohmatsu India LLP

Faculty Supervisor
Dr. Moon Inder Singh
Associate Professor
DEIE, TIET, Patiala



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

2026

Department of Electrical and Instrumentation Engineering
Thapar Institute of Engineering and Technology, Patiala
(Declared as Deemed-to-be-University u/s 3 of the UGC Act., 1956)
Post Bag No. 32, Patiala – 147004
Punjab (India)

Declaration

I hereby declare that the final report titled "Development of Smart-Cart Application Using Java Spring Boot", submitted as a partial requirement for the Project Semester (UEE894) course towards the Bachelor of Engineering degree in Electrical & Computer Engineering at Thapar Institute of Engineering and Technology, Patiala, is an accurate representation of the work undertaken during my internship at Deloitte Touche Tohmatsu India LLP.

This work was carried out under the professional guidance of my Host Mentor at Deloitte India (Technology & Transformation Practice) and under the academic supervision of Mr. Moon Inder Singh, Assistant Professor, Department of Electrical and Instrumentation Engineering, TIET, Patiala.

I further declare that the work reported in this report has not been submitted, either in part or full, for the award of any other degree or diploma at this or any other institution.

Place: Patiala

Student Name: Yash Bansal

Date: _____

Roll No.: 102219073

It is certified that the above statement made by the student is correct to the best of my knowledge and belief.

Host Mentor

Confidential
Technology & Transformation Practice
Deloitte Touche Tohmatsu India LLP

Faculty Supervisor

Dr. Moon Inder Singh
Associate Professor
DEIE, TIET, Patiala

Acknowledgement

I would like to express my sincere gratitude to Deloitte Touche Tohmatsu India LLP for providing me the opportunity to work as an intern within the Technology & Transformation – Marketing and Commerce practice. This internship has been an invaluable experience that enabled me to apply academic knowledge in real-world enterprise technology environments and significantly strengthened my professional engineering capabilities.

I extend my heartfelt thanks to my Host Mentor and all team members at Deloitte Touche Tohmatsu India LLP for their continuous guidance, technical mentorship, and constant encouragement throughout the six-month internship. Their support helped me understand enterprise software development practices, collaborative workflows using Git and GitHub, structured code review processes, and professional engineering standards in large-scale client engagements.

I am deeply grateful to Dr. Moon Inder Singh, Associate Professor, Department of Electrical and Instrumentation Engineering, TIET, Patiala, for his unwavering academic supervision and encouragement during the project semester. His timely feedback and guidance have been instrumental in shaping this report.

I also thank Thapar Institute of Engineering and Technology for structuring the Project Semester (UEE894) curriculum in a manner that bridges academic learning with meaningful industrial exposure.

Yash Bansal

Abstract

This final report documents the complete work undertaken during a six-month project semester internship at Deloitte Touche Tohmatsu India LLP, within the Technology & Transformation (Marketing and Commerce) practice, from January 23, 2026. The internship focused on enterprise software engineering, digital commerce platform development, and backend systems design using modern technologies.

The primary technical contribution of the internship was the design and development of a Smart-Cart application using Java Spring Boot for a confidential client engagement. Responsibilities encompassed active participation across multiple phases of the Software Development Life Cycle (SDLC), including requirements gathering, application architecture design, backend development, security hardening, testing, and deployment-related activities. The application was engineered to be resilient against common cybersecurity vulnerabilities, with particular attention to secure coding practices. Version control was managed using Git and GitHub, involving the creation, management, and merging of multiple feature branches through structured development workflows.

In addition to the Smart-Cart project, the internship included development of full-stack web applications using React and Angular, implementation of an end-to-end payment backend system supporting recurring auto-pay workflows, exposure to distributed system technologies including Apache Kafka and Elasticsearch, and completion of structured learning programs in Generative AI, cybersecurity awareness, and professional ethics.

By the conclusion of the internship, significant technical proficiency was achieved in backend architecture, REST API development, database management using PostgreSQL, and enterprise-level software engineering practices. This report synthesizes the objectives, methodology, technical contributions, challenges encountered, key learnings, and outcomes of the internship, culminating in a comprehensive reflection on professional and academic growth.

Table of Contents

Declaration	i
Acknowledgement	ii
Abstract	iii
Table of Contents	iv
List of Tables	v
List of Abbreviations	vi
Assumptions Made	vii
Chapter 1: Introduction	1
1.1 Industry and Organizational Background	1
1.2 Problem Statement	2
1.3 Internship Objectives	3
1.4 Scope of Work	3
Chapter 2: Review of Technologies and Concepts	5
2.1 Frontend Development Technologies	5
2.2 Backend Development: Java Spring Boot	5
2.3 Smart-Cart Application Architecture	6
2.4 Security and Cybersecurity Practices	7
2.5 Version Control with Git and GitHub	7
2.6 Database Management with PostgreSQL	8
2.7 Distributed Systems and Scalable Technologies	8
2.8 Generative AI and Professional Training	9
Chapter 3: Methodology	10
3.1 SDLC Participation	10
3.2 Development Workflow	11
3.3 Collaboration and Version Control Practices	12
3.4 Testing, Debugging, and Maintenance	12
Chapter 4: Work Done and Technical Contributions	13
4.1 Smart-Cart Application Development	13
4.2 Payment Backend System	15

4.3 E-Commerce Web Application	16
4.4 Angular Enterprise Module Development	16
4.5 Distributed Systems Exposure	17
4.6 AI and Professional Training Programs	17
Chapter 5: Progress Assessment and Key Learnings	18
5.1 Goal-Wise Progress Assessment	18
5.2 Technical Skills Acquired	19
5.3 Challenges Faced and Overcome	20
5.4 Professional Development	20
Chapter 6: Conclusions and Future Work	21
6.1 Conclusions	21
6.2 Future Work	22
References	23
Appendix A – Project Timeline	24
Appendix B – List of Key Tools and Technologies	25
Plagiarism Report	26

List of Tables

Table No.	Title	Page
Table 3.1	Project Phases and SDLC Activities	11
Table 4.1	Smart-Cart Application Technology Stack	14
Table 5.1	Goal-Wise Progress Assessment	18
Table 5.2	Technical Skills Acquired During Internship	19

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
ECE	Electrical & Computer Engineering
GenAI	Generative Artificial Intelligence
Git	Distributed Version Control System
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JWT	JSON Web Token
Kafka	Apache Kafka (Distributed Event Streaming Platform)
LLM	Large Language Model
MVC	Model View Controller
ORM	Object Relational Mapping
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SQL	Structured Query Language
TIET	Thapar Institute of Engineering and Technology
UI	User Interface
UX	User Experience

Chapter 1

Introduction

Chapter Outline

This chapter introduces the industrial and organizational context of the internship at Deloitte Touche Tohmatsu India LLP, describes the engineering problem addressed, outlines the core objectives of the project semester, and defines the overall scope of work undertaken during the six-month engagement.

1.1 Industry and Organizational Background

Modern enterprises are rapidly transitioning toward digital-first business models where scalable software systems form the backbone of customer engagement, operational efficiency, and revenue generation. Organizations require robust digital commerce platforms that support high transaction volumes, seamless user experiences, secure payment processing, and scalable backend architectures. In this context, software engineering expertise — particularly in backend development and full-stack application design — has become a critical competency for technology professionals.

Deloitte Touche Tohmatsu India LLP is one of the leading professional services organizations in India, providing consulting, audit, risk advisory, financial advisory, and technology services across multiple industries. Its Technology & Transformation practice focuses on helping enterprises modernize their digital ecosystems by building scalable, secure, and customer-centric technology platforms. The Marketing and Commerce domain within this practice specifically supports organizations in developing digital platforms such as e-commerce systems, customer engagement portals, and enterprise web applications.

The internship was conducted within the Technology & Transformation – Marketing and Commerce practice, providing hands-on exposure to enterprise software development workflows, client-focused solution delivery, and professional engineering standards in a large-scale consulting environment.

1.2 Problem Statement

Enterprise digital commerce platforms face several interconnected engineering challenges that demand proficiency across multiple layers of software architecture:

- Building scalable and maintainable backend systems capable of handling high transaction volumes.
- Designing secure application architectures resilient to common cybersecurity vulnerabilities.
- Developing modular frontend systems that deliver responsive and consistent user experiences.

- Designing robust payment architectures that support recurring billing and lifecycle management.
- Integrating APIs efficiently across multiple internal and third-party services.
- Managing structured data persistently through optimized database schemas.
- Collaborating effectively in distributed, remote development teams using version control systems.

The primary engineering challenge addressed during this internship was the development of a Smart-Cart application using Java Spring Boot for a client engagement within Deloitte India. The Smart-Cart application required careful attention to backend architecture design, security implementation, and alignment with the Software Development Life Cycle (SDLC) standards practiced at the enterprise level.

Beyond the Smart-Cart project, the internship aimed to bridge the gap between academic preparation and industry-level engineering practices by developing competencies across full-stack development, distributed systems, and professional collaboration workflows.

1.3 Internship Objectives

The internship was structured around six core objectives, as established in the Goal Report:

- Goal 1 — Full-Stack Engineering Skills: Develop practical experience in building end-to-end applications using React, Angular, Java, and Spring Boot.
- Goal 2 — Backend Architecture Development: Build backend engineering expertise through the creation of payment systems, recurring transaction workflows, service-layer architecture, and REST APIs.
- Goal 3 — API Integration Capability: Learn how enterprise systems communicate through third-party APIs, internal APIs, authentication workflows, and service integration patterns.
- Goal 4 — Database Management: Develop understanding of structured data systems using PostgreSQL, including query optimization and data persistence design.
- Goal 5 — Scalable System Learning: Gain exposure to distributed systems tools such as Apache Kafka and Elasticsearch.
- Goal 6 — AI and Emerging Technology Exposure: Explore emerging technologies including Python, OpenCV, and Generative Artificial Intelligence.

1.4 Scope of Work

The internship commenced on January 23, 2026, and spans six months in remote mode. The scope of work encompassed three broad phases:

1.4.1 Learning and Onboarding Phase

The initial weeks were devoted to onboarding, understanding project requirements, studying enterprise workflows, reviewing existing codebases, and learning internal development practices and collaboration standards followed at Deloitte India.

1.4.2 Development Phase

This phase constituted the core of the internship and involved active development work across multiple projects, including the Smart-Cart backend application (Java Spring Boot), full-stack e-commerce web application (React), enterprise module development (Angular), and payment backend systems. This phase also involved participation in structured SDLC activities, including requirements analysis, architecture design, code reviews, and version control management using Git and GitHub.

1.4.3 Advanced Engineering and Consolidation Phase

The latter phase of the internship focused on deepening backend optimization skills, understanding scalable architecture patterns, improving production-level engineering practices (debugging, testing, and deployment), and exploring enterprise applications of Generative AI. Completion of Deloitte internal professional training programs — covering ethics, cybersecurity, and organizational compliance — also occurred during this phase.

Chapter 2

Review of Technologies and Concepts

Chapter Outline

This chapter reviews the core technologies and conceptual frameworks employed during the internship, providing the technical foundation necessary to understand the work described in subsequent chapters. It covers frontend technologies, backend systems, the Smart-Cart application architecture, security practices, version control, database management, distributed systems, and professional training programs.

2.1 Frontend Development Technologies

Frontend development work involved building responsive and modular user interfaces using industry-standard technologies:

- **HTML and CSS:** Employed for structured document presentation and responsive visual design, forming the foundational layer of web application development.
- **React:** A component-based JavaScript library used for building dynamic and reusable user interface elements. React's virtual DOM and declarative programming model make it well-suited for large-scale frontend applications requiring frequent UI updates.
- **Angular:** A TypeScript-based enterprise frontend framework maintained by Google, widely used in enterprise-grade applications for its opinionated architecture, robust module system, dependency injection, and built-in support for routing, data binding, and form validation.

These technologies emphasize modular architecture, reusable components, and maintainable code structures — all of which are core requirements in enterprise-level development environments.

2.2 Backend Development: Java Spring Boot

Java Spring Boot served as the primary backend technology throughout the internship, particularly for the Smart-Cart application. Spring Boot is an open-source, Java-based framework that simplifies the development of production-ready applications by providing embedded server configurations, auto-configuration, and an extensive ecosystem of libraries.

Key features and capabilities of Spring Boot utilized during the internship include:

- **RESTful API Design:** Development of REST APIs following standard HTTP verb conventions (GET, POST, PUT, DELETE), enabling clean communication between frontend clients and backend services.
- **Spring Security:** Implementation of security layers including authentication, authorization, and protection against common vulnerabilities such as Cross-Site Request Forgery (CSRF), SQL injection, and unauthorized access.

- Spring Data JPA: Object-Relational Mapping (ORM) framework used for managing database interactions with minimal boilerplate code, enabling clean persistence layer design.
- Service-Layer Architecture: Application logic was organized into clearly separated controller, service, and repository layers following the MVC (Model-View-Controller) design pattern, improving code modularity and testability.
- Dependency Injection: Spring's inversion-of-control container was leveraged to manage object lifecycles and promote loosely coupled component design.

2.3 Smart-Cart Application Architecture

The Smart-Cart application was designed as a backend-centric e-commerce solution intended to provide core shopping cart functionality for a client engagement. The application architecture was informed by enterprise software engineering principles, with careful attention to scalability, maintainability, and security.

The core architectural layers of the Smart-Cart application comprised:

- Presentation Layer: REST API endpoints exposed to frontend consumers, designed following RESTful conventions to ensure platform-agnostic integration.
- Business Logic Layer: Service classes encapsulating core cart operations such as item addition, removal, quantity management, discount calculation, and order finalization.
- Persistence Layer: Repository interfaces managing communication with the underlying relational database, implemented using Spring Data JPA.
- Security Layer: Authentication and authorization mechanisms implemented using Spring Security to restrict access based on user roles and prevent exploitation of common web vulnerabilities.

Version control for the Smart-Cart project was managed using Git and GitHub, with structured branch management involving feature branches, pull requests, code reviews, and controlled merges into the main development branch.

2.4 Security and Cybersecurity Practices

A critical component of the Smart-Cart application development was the implementation of security measures to make the application resilient against common cyber attacks and vulnerabilities. Security considerations were addressed at multiple layers of the application:

- Input Validation and Sanitization: All user inputs were validated at the API layer to prevent injection attacks, including SQL injection and Cross-Site Scripting (XSS).
- Authentication and Authorization: Role-based access control was implemented to ensure that only authorized users could perform sensitive operations such as order placement and cart modification.
- Secure API Design: APIs were designed to minimize information disclosure, enforce proper error handling, and prevent unauthorized data exposure.

- **Dependency Management:** Security-conscious management of third-party library versions to avoid known vulnerabilities in dependencies.

In addition, completion of Deloitte's internal Cybersecurity Awareness training program reinforced theoretical knowledge of secure software development practices within a professional organizational context.

2.5 Version Control with Git and GitHub

Structured version control was a central practice throughout the internship. Git was used as the distributed version control system, with GitHub serving as the remote repository hosting platform. Key practices included:

- **Branch Strategy:** Separate feature branches were created for individual development tasks, ensuring isolation of work in progress from the stable main branch.
- **Pull Requests and Code Reviews:** All completed features were submitted via pull requests, enabling peer review, discussion, and quality validation before integration into the primary codebase.
- **Merge Management:** Branches were merged following review approval, with attention to resolving merge conflicts and ensuring codebase consistency.
- **Commit Practices:** Descriptive and meaningful commit messages were maintained to ensure traceability of changes across the development history.

2.6 Database Management with PostgreSQL

PostgreSQL was the primary relational database management system utilized during the internship. Key database-related activities included:

- **Schema Design:** Designing normalized database schemas to efficiently model entities such as users, products, cart items, and orders.
- **CRUD Operations:** Implementing Create, Read, Update, and Delete operations through Spring Data JPA repository interfaces.
- **Query Optimization:** Writing and analyzing SQL queries to improve data retrieval performance in data-intensive operations.
- **Data Persistence Design:** Planning data lifecycle management strategies to ensure consistency, integrity, and reliability of stored information.

2.7 Distributed Systems and Scalable Technologies

Exposure to enterprise-scale distributed systems tools was an important component of the internship's advanced engineering phase:

- **Apache Kafka:** An open-source distributed event streaming platform used for building real-time data pipelines and event-driven microservice architectures. Kafka enables decoupled communication between services through a publish-subscribe messaging model, providing high throughput and fault tolerance.

- Elasticsearch: A distributed search and analytics engine built on Apache Lucene, enabling full-text search, log analysis, and real-time indexing. In enterprise commerce platforms, Elasticsearch is commonly used for product search optimization and analytics.

These technologies provided conceptual grounding in scalable system design patterns applicable to high-traffic digital commerce platforms of the kind handled within Deloitte's client engagements.

2.8 Generative AI and Professional Training

Self-directed learning during the internship included the completion of a structured 17-hour Generative AI course covering:

- Large Language Model (LLM) concepts and architecture fundamentals.
- Prompt engineering strategies for effective use of AI-assisted tools.
- AI-assisted development workflows and their applicability in enterprise software engineering.

Additionally, the following Deloitte internal training programs were completed, collectively reinforcing responsible and professional engineering conduct:

- Ethics and Professional Conduct: Understanding Deloitte's code of professional ethics and responsibilities in client engagements.
- Cybersecurity Awareness: Practical knowledge of common cybersecurity threats and organizational best practices for data protection.
- Organizational Compliance Training: Familiarization with internal policies, procedures, and regulatory requirements governing professional behavior within the firm.

Chapter 3

Methodology

Chapter Outline

This chapter describes the methodological approach followed during the internship, encompassing participation in the Software Development Life Cycle (SDLC), the development workflow adopted for the Smart-Cart project and other assignments, collaboration and version control practices, and the testing and maintenance strategies employed.

3.1 SDLC Participation

One of the defining responsibilities of the internship was active participation in and ownership of major phases of the Software Development Life Cycle. The SDLC framework followed during the internship was broadly aligned with Agile principles, emphasizing iterative development, continuous feedback, and incremental delivery. The key SDLC phases undertaken are described below.

3.1.1 Requirements Understanding and Analysis

Each development task began with a structured requirements understanding phase. This involved reviewing client briefs and user stories, clarifying functional and non-functional requirements with senior engineers and the host mentor, and decomposing high-level requirements into actionable development tasks. For the Smart-Cart application, this phase included analyzing the expected cart behaviors, security requirements, and integration touchpoints with other system components.

3.1.2 Application Architecture and Design

Following requirements clarity, application architecture and design decisions were made for the Smart-Cart application. This included selecting the appropriate layered architecture (Controller-Service-Repository), defining API contracts and data models, planning the security framework, and establishing conventions for code organization and naming. Design decisions were made with the goals of maintainability, testability, and scalability in mind.

3.1.3 Backend Development

The core development phase involved implementing the defined requirements using Java Spring Boot. RESTful API endpoints were developed, business logic was encapsulated in service classes, and persistence was managed through Spring Data JPA repositories connected to a PostgreSQL database. Security measures were implemented at both the API and service layers.

3.1.4 Testing and Debugging

Unit testing and integration testing were conducted to validate the correctness of implemented features. Debugging activities addressed both logical defects identified during testing and issues

discovered during peer code reviews. Testing was performed iteratively throughout the development phase rather than exclusively at its conclusion.

3.1.5 Deployment-Related Activities

Deployment-related tasks included preparing the application for integration into the broader development environment, managing configuration properties for different deployment targets, and ensuring that the application startup and runtime behaviors were consistent and reliable.

Table 3.1: Project Phases and SDLC Activities		
SDLC Phase	Key Activities	Technologies/Tools
Requirements Analysis	User story review, task decomposition, API contract definition	Confluence, team meetings
Architecture Design	Layer design, data model planning, security framework selection	UML diagrams, Spring Boot
Backend Development	API development, business logic, persistence layer, security	Java, Spring Boot, PostgreSQL
Testing & Debugging	Unit tests, integration tests, defect resolution, code reviews	JUnit, debugging tools
Version Control	Branch management, pull requests, code reviews, merges	Git, GitHub
Deployment Activities	Configuration management, environment setup, runtime validation	Spring Boot profiles, command-line tools

3.2 Development Workflow

The day-to-day development workflow followed a continuous learning and delivery model. For each development task, the approach consisted of:

- Requirement Understanding: Reviewing the task requirements, clarifying ambiguities with the mentor or team, and planning the implementation approach.
- Implementation: Writing clean, well-documented code adhering to the coding standards and architecture conventions established for the project.
- Self-Review: Reviewing one's own implementation for correctness, completeness, and adherence to design intent before submission.
- Code Review and Iteration: Participating in peer code reviews, incorporating feedback, and refining the implementation accordingly.
- Integration: Merging approved changes into the development branch following the pull request workflow on GitHub.

3.3 Collaboration and Version Control Practices

Collaboration in the remote internship environment was supported by structured version control and communication practices. Git branches were created for each feature or task, with descriptive branch names reflecting the scope of the work. Completed branches were submitted via pull requests on GitHub, where team members could review the code, leave comments, and approve or request changes. Approved pull requests were then merged into the designated integration branch.

This workflow ensured that the main codebase remained stable and that all changes were subject to quality validation before integration. Management of multiple concurrent branches — including creating, reviewing, and merging branches — was a consistent responsibility throughout the internship.

3.4 Testing, Debugging, and Maintenance

Testing activities during the internship encompassed both unit-level validation of individual components and integration testing of assembled features. Debugging was approached systematically: identifying the symptoms, isolating the root cause, formulating and applying a fix, and verifying that the fix resolved the issue without introducing regressions. Maintenance activities included updating existing functionality in response to changing requirements, refactoring code for improved clarity or performance, and resolving defects identified post-integration.

Chapter 4

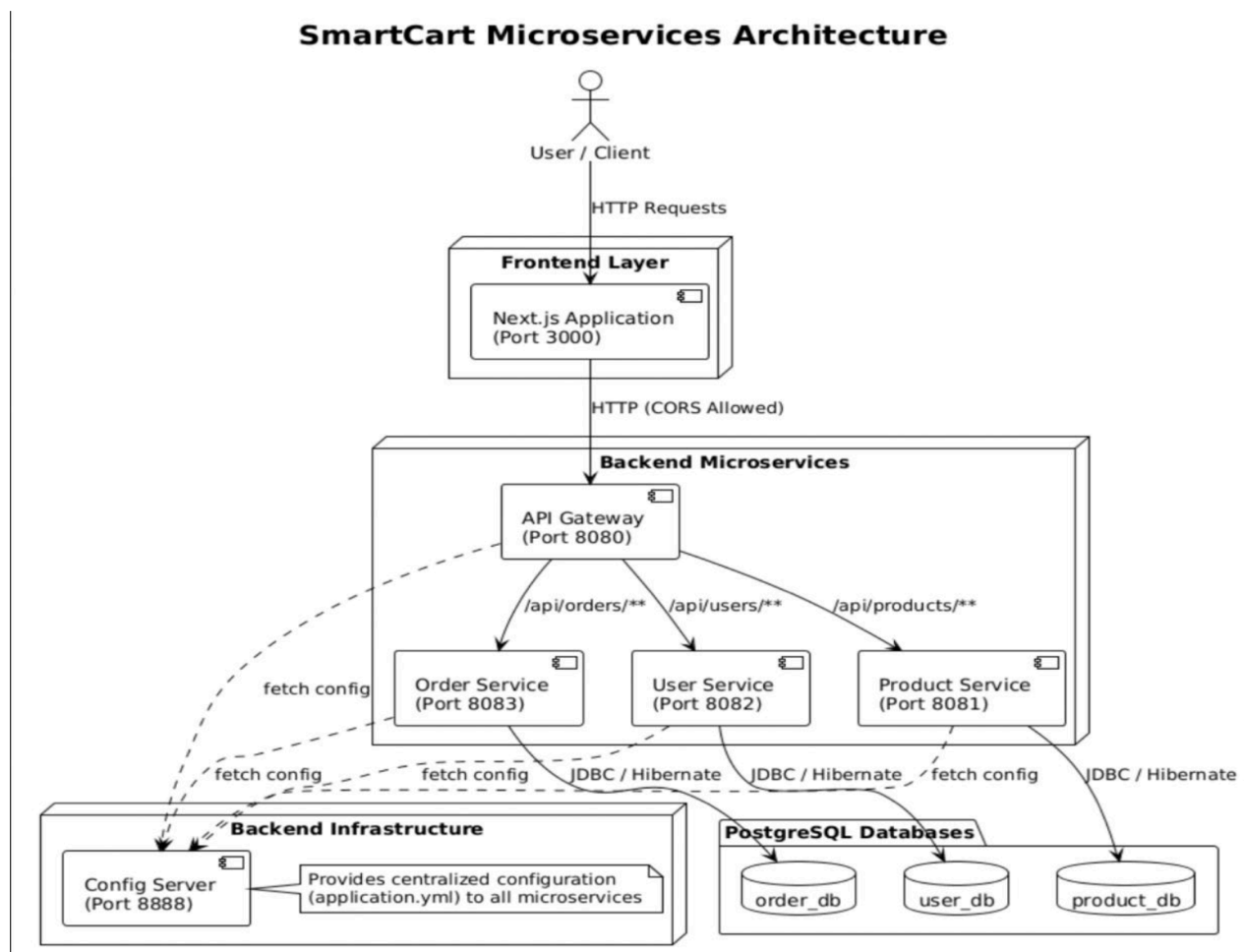
Work Done and Technical Contributions

Chapter Outline

This chapter presents a detailed account of the technical work completed during the internship, organized by project and area of contribution. It covers the Smart-Cart application development, payment backend system, e-commerce web application, Angular module development, distributed systems exposure, and AI and professional training programs.

4.1 Smart-Cart Application Development

The development of the Smart-Cart application using Java Spring Boot constituted the primary technical contribution of the internship. This was a client-focused backend development engagement conducted within the Deloitte Technology & Transformation practice. Responsibilities spanned the full breadth of the Software Development Life Cycle, from requirements understanding through to deployment-related activities.



4.1.1 Requirements Understanding and Feature Design

The Smart-Cart application's functional requirements were analyzed to understand the expected behaviors of a shopping cart system. This included identifying the entities involved (users, products, cart items, orders), the operations to be supported (item addition, quantity updates, removal, checkout), and the non-functional requirements pertaining to performance, security, and maintainability. Feature design involved defining the API contract — specifying endpoints, request/response schemas, and HTTP methods — prior to implementation.

4.1.2 Backend Development and API Implementation

The backend was developed using Java Spring Boot following a layered MVC architecture. RESTful APIs were implemented to expose cart management operations to client-side consumers. Controller classes handled HTTP requests and responses, service classes encapsulated the business logic, and repository interfaces managed database interactions through Spring Data JPA connected to a PostgreSQL database.

Table 4.1: Smart-Cart Application Technology Stack		
Component	Technology	Purpose
Backend Framework	Java Spring Boot	API and business logic
ORM / Persistence	Spring Data JPA	Database interaction
Database	PostgreSQL	Relational data storage
Security	Spring Security	Auth & vulnerability protection
API Style	RESTful (HTTP/JSON)	Client-server communication
Version Control	Git / GitHub	Source management & collaboration

4.1.3 Security Implementation

A dedicated effort was made to make the Smart-Cart application resilient against common cyber attacks and vulnerabilities. Security was implemented at multiple levels using Spring Security. Authentication mechanisms were established to ensure that only legitimate users could access cart operations. Authorization controls were applied to restrict sensitive operations to users with appropriate roles. Input validation was enforced at the API boundary to prevent injection-based attacks. API responses were carefully designed to avoid exposing sensitive internal data or system information. These measures collectively ensured that the application met the security expectations of an enterprise client engagement.

4.1.4 Version Control and Branch Management

The Smart-Cart project was managed under a Git-based version control system hosted on GitHub. A structured branching strategy was followed, with separate feature branches created for each development task. Upon completion of each feature, a pull request was raised for peer review. Code review feedback was incorporated iteratively, and approved branches were merged into the main development branch. Multiple branches were managed concurrently, and merge conflicts were resolved methodically to maintain codebase integrity.

4.1.5 Testing, Debugging, and Maintenance

Unit tests were written to validate the behavior of individual service and repository components. Integration tests were used to verify the correctness of API endpoints and their interactions with the persistence layer. Defects identified during testing or code review were analyzed, traced to their root cause, and resolved with appropriate fixes. Maintenance activities included refactoring sections of code for improved clarity and updating feature implementations to reflect revised requirements.

4.2 Payment Backend System

A significant backend development project during the internship involved the implementation of a payment backend system using Java Spring Boot. This system was designed to handle the complete transaction lifecycle for a digital commerce platform, supporting both one-time payment processing and recurring auto-pay workflows.

Key functionalities implemented in the payment backend system included:

- **Transaction Lifecycle Handling:** Managing the complete lifecycle of a payment transaction, from initiation through processing, confirmation, and failure handling.
- **Recurring Subscription Payments:** Implementing logic to support recurring billing arrangements, where customers authorize periodic automatic charges at defined intervals.
- **Service-Layer Abstraction:** Designing clean service-layer interfaces that decoupled payment processing logic from the API layer, enabling testability and future extensibility.
- **API-Based Integration:** Exposing payment operations through REST APIs, enabling integration with frontend consumers and third-party payment gateways.
- **Validation and Error Handling:** Implementing robust input validation and structured error response mechanisms to ensure reliable operation under edge cases and invalid inputs.

This project provided substantial exposure to the complexities of enterprise payment workflows, including the handling of transactional consistency, error recovery, and secure data management.

4.3 E-Commerce Web Application

A full-stack e-commerce web application was developed as part of a client engagement. The frontend was built using React, following a component-based architecture that emphasized modularity, reusability, and maintainability.

Key contributions to the e-commerce application included:

- **Responsive UI Design:** Development of responsive user interface components that adapt to different screen sizes, ensuring a consistent experience across desktop and mobile devices.
- **Product Listing Components:** Implementation of dynamic product listing pages capable of rendering variable product catalogs fetched from backend APIs.
- **User Interaction Workflows:** Building user-facing workflows including product browsing, cart interaction, and checkout initiation.
- **Modular React Architecture:** Organizing the frontend codebase into reusable, independently maintainable React components consistent with enterprise frontend development standards.

4.4 Angular Enterprise Module Development

Angular was used to develop structured enterprise-grade web modules within existing enterprise web systems. This work provided practical exposure to large-scale enterprise frontend framework usage in a professional environment.

Activities performed during Angular module development included:

- **Component Creation:** Developing Angular components to encapsulate specific UI behaviors and data presentation logic.
- **Data Binding Implementation:** Utilizing Angular's two-way data binding capabilities to synchronize UI state with underlying application data models.
- **Routing Configuration:** Configuring Angular's routing module to manage navigation between different views within the enterprise application.
- **UI Validation Workflows:** Implementing form validation logic to ensure data integrity prior to submission, using Angular's reactive forms module.

4.5 Distributed Systems Exposure

The internship provided structured exposure to distributed systems technologies used in enterprise-scale digital platforms:

- **Apache Kafka:** Kafka's event-driven communication model, producer-consumer architecture, and topic-based message routing were studied and explored. Understanding Kafka's role in decoupling microservices and enabling asynchronous communication provided valuable architectural insight applicable to high-volume digital commerce systems.
- **Elasticsearch:** Elasticsearch's indexing model, query DSL, and integration patterns within backend systems were explored. Its application in product search optimization and real-time analytics was examined in the context of the commerce platform.

4.6 AI and Professional Training Programs

In addition to hands-on technical development, the internship included the completion of the following structured learning programs:

- Generative AI Course (17 hours): Covering Large Language Model fundamentals, prompt engineering techniques, and practical AI-assisted development workflows relevant to enterprise software engineering.
- Ethics and Professional Conduct Training: Reinforcing professional responsibilities, client confidentiality obligations, and conduct standards applicable within a consulting organization.
- Cybersecurity Awareness Training: Covering common threat vectors, organizational security protocols, and best practices for protecting sensitive data and systems.
- Organizational Compliance Training: Familiarization with Deloitte's internal policies, regulatory compliance requirements, and procedural guidelines.

Chapter 5

Progress Assessment and Key Learnings

Chapter Outline

This chapter evaluates the progress achieved against the original internship goals, summarizes the technical and professional skills acquired, and reflects on the challenges encountered and overcome during the six-month project semester.

5.1 Goal-Wise Progress Assessment

Table 5.1 provides a structured assessment of progress against each of the six goals defined at the commencement of the internship.

Table 5.1: Goal-Wise Progress Assessment		
Goal	Status	Key Achievements
Full-Stack Engineering Skills	Achieved	Built e-commerce app (React), enterprise modules (Angular), backend (Spring Boot)
Backend Architecture Development	Achieved	Developed Smart-Cart app, payment backend with lifecycle management and auto-pay
API Integration Capability	Achieved	REST API development, service integration, authentication workflows
Database Management	Achieved	PostgreSQL schema design, CRUD operations, query optimization using JPA
Scalable System Learning	Achieved	Conceptual and practical exposure to Apache Kafka and Elasticsearch
AI and Emerging Technology	Achieved	Completed 17-hour GenAI course; explored AI-assisted development workflows

5.2 Technical Skills Acquired

Table 5.2 summarizes the key technical skills developed during the internship, organized by domain.

Table 5.2: Technical Skills Acquired During Internship	
Domain	Skills Acquired
Backend Development	Spring Boot REST API design, service-layer architecture, JPA-based persistence, Spring Security
Frontend Development	React component design, Angular module development, responsive UI, data binding, routing

Security Engineering	Input validation, authentication & authorization, secure API design, cybersecurity awareness
Database Management	PostgreSQL schema design, CRUD operations, SQL query optimization, JPA repositories
Version Control	Git branching strategy, pull requests, code reviews, merge management, conflict resolution
SDLC Practices	Requirements analysis, architecture design, iterative development, testing, deployment
Distributed Systems	Apache Kafka event streaming concepts, Elasticsearch indexing and search
Artificial Intelligence	LLM fundamentals, prompt engineering, AI-assisted development workflows

5.3 Challenges Faced and Overcome

The internship presented a range of technical and professional challenges that contributed significantly to engineering maturity and adaptability:

5.3.1 Transition from Academic to Enterprise Workflows

The shift from structured academic assignments to open-ended enterprise development tasks required rapid adaptation. Unlike academic projects with well-defined problem statements, real-world engineering tasks involved ambiguous requirements, evolving specifications, and collaborative dependencies. This challenge was addressed by developing a habit of proactive requirement clarification and incremental delivery.

5.3.2 Navigating Large Enterprise Codebases

Understanding and contributing to large, existing codebases — particularly during the e-commerce and Angular module development projects — required systematic code reading, documentation review, and progressive familiarization. The ability to work within established code structures without disrupting existing functionality was developed progressively over the internship.

5.3.3 Managing Multiple Technologies Simultaneously

The breadth of technologies employed concurrently — spanning Java Spring Boot, React, Angular, PostgreSQL, Git, Kafka, and Elasticsearch — required efficient context-switching and concurrent learning. This challenge was managed through prioritization based on project relevance and structured self-learning supplemented by internal Deloitte training resources.

5.3.4 Remote Collaboration

Operating in a fully remote environment necessitated disciplined use of collaboration tools, clear written communication, and proactive status updates to maintain alignment with the team. These practices were developed and refined over the course of the internship.

5.3.5 Security Implementation Complexity

Designing and implementing security measures for the Smart-Cart application required understanding not only the Spring Security framework but also the broader landscape of common web vulnerabilities and corresponding mitigation strategies. This challenge was addressed through systematic study of Spring Security documentation, cybersecurity training, and iterative implementation with mentor feedback.

5.4 Professional Development

Beyond technical skill acquisition, the internship contributed substantially to professional growth. Working within Deloitte's Technology & Transformation practice provided exposure to:

- Enterprise engineering culture: Understanding the standards, practices, and expectations that govern professional software development in a large consulting organization.
- Client-focused delivery: Developing an orientation toward producing solutions that meet client requirements reliably, securely, and within expected timelines.
- Structured collaboration: Practicing disciplined use of version control workflows, code review processes, and communication conventions that support effective team collaboration.
- Ethical engineering responsibility: Internalizing the importance of ethics, data protection, and regulatory compliance in technology practice through formal training programs.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

The six-month project semester internship at Deloitte Touche Tohmatsu India LLP, within the Technology & Transformation (Marketing and Commerce) practice, has been a comprehensive and transformative professional experience. The internship achieved all six goals defined at its commencement, delivering meaningful technical contributions across backend development, full-stack engineering, payment systems architecture, and enterprise web application development.

The primary technical achievement of the internship was the development of the Smart-Cart application using Java Spring Boot. This project engaged all phases of the Software Development Life Cycle and required the application of backend engineering principles, secure architecture design, REST API development, PostgreSQL-based persistence, and structured version control practices using Git and GitHub. Security was treated as a first-class concern throughout the development process, with the application designed to be resilient against common web vulnerabilities through the application of Spring Security and rigorous input validation practices.

In addition to the Smart-Cart project, significant contributions were made to full-stack web application development (React and Angular), payment backend system implementation (Spring Boot), and distributed systems learning (Apache Kafka and Elasticsearch). Structured professional training programs in Generative AI, cybersecurity, ethics, and organizational compliance further strengthened readiness for a professional engineering career.

The challenges encountered during the internship — including navigating large enterprise codebases, managing multiple technologies concurrently, adapting to remote collaboration environments, and implementing enterprise-grade security — were addressed through disciplined learning, mentorship, and iterative professional growth. These challenges ultimately accelerated development into a more capable, adaptable, and enterprise-ready software engineer.

Overall, this internship successfully bridged the gap between academic preparation and industry practice, equipping the student with the technical competencies, professional practices, and engineering mindset necessary for a career in software engineering, backend development, and full-stack application design.

6.2 Future Work

Building on the foundations established during this internship, the following directions are identified for continued growth and development:

- **Advanced Spring Boot Architecture:** Deepening expertise in advanced Spring Boot patterns such as reactive programming (Spring WebFlux), microservice decomposition, and cloud-native deployment strategies.
- **Microservices and Container Orchestration:** Gaining practical experience with containerization (Docker) and orchestration (Kubernetes) to manage scalable, distributed microservice deployments.
- **Enhanced Security Engineering:** Expanding security knowledge to include OAuth 2.0/OIDC authentication flows, penetration testing fundamentals, and compliance frameworks such as OWASP Top 10 mitigation strategies.
- **Production-Level DevOps Practices:** Developing skills in CI/CD pipeline design and implementation, enabling automated testing, build, and deployment workflows.
- **Kafka and Elasticsearch Integration:** Transitioning from conceptual exposure to hands-on implementation of Kafka-based event streaming and Elasticsearch-powered search features within production-quality applications.
- **AI Integration in Backend Systems:** Exploring the practical integration of Large Language Models and AI-driven features into enterprise backend applications, leveraging the foundational GenAI knowledge acquired during the internship.
- **Smart-Cart Feature Extension:** Future iterations of the Smart-Cart application could incorporate recommendation engine integration, real-time inventory management, multi-currency support, and enhanced analytics capabilities.

References

- [1] G. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [2] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.
- [3] Spring Framework Documentation — Spring Boot Reference Guide, Pivotal Software, 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [4] Spring Security Reference Documentation, Pivotal Software, 2024. [Online]. Available: <https://docs.spring.io/spring-security/reference/>
- [5] PostgreSQL Documentation, The PostgreSQL Global Development Group, 2024. [Online]. Available: <https://www.postgresql.org/docs/>
- [6] Apache Kafka Documentation, Apache Software Foundation, 2024. [Online]. Available: <https://kafka.apache.org/documentation/>
- [7] Elasticsearch Reference Documentation, Elastic N.V., 2024. [Online]. Available: <https://www.elastic.co/guide/en/elasticsearch/reference/current/>
- [8] React Documentation, Meta Open Source, 2024. [Online]. Available: <https://react.dev/>
- [9] Angular Documentation, Google LLC, 2024. [Online]. Available: <https://angular.dev/>
- [10] OpenAPI Specification, SmartBear Software, 2024. [Online]. Available: <https://swagger.io/specification/>
- [11] OWASP Top Ten, Open Web Application Security Project, 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [12] Git Reference Documentation, Software Freedom Conservancy, 2024. [Online]. Available: <https://git-scm.com/docs>
- [13] Deloitte India — Technology & Transformation Practice Overview, Deloitte Touche Tohmatsu India LLP, 2024. [Online]. Available: <https://www2.deloitte.com/in/en.html>

Appendix A – Project Timeline

The following table outlines the planned project timeline for the six-month internship, as defined in the Goal Report and tracked through the midway period.

Task / Activity	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6
Onboarding & Orientation	✓					
Smart-Cart Backend Development		✓	✓	✓		
Frontend Development (React, Angular)		✓	✓	✓		
Payment Backend System			✓	✓		
API Integration Work		✓	✓	✓	✓	
Database Work (PostgreSQL)		✓	✓	✓		
Distributed Systems Learning				✓	✓	✓
AI & Professional Training		✓	✓	✓		
Final Report Preparation					✓	✓

Appendix B – List of Key Tools and Technologies

The following table summarizes the key tools and technologies utilized across the duration of the internship.

Tool / Technology	Category	Usage During Internship
Java	Programming Language	Primary backend development language
Spring Boot	Backend Framework	Smart-Cart app development, payment backend, REST APIs
Spring Security	Security Framework	Authentication, authorization, vulnerability mitigation
Spring Data JPA	ORM Library	Database entity management and repository layer
PostgreSQL	Database	Relational data storage for application entities
React	Frontend Framework	E-commerce web application UI development
Angular	Frontend Framework	Enterprise web module development
Git	Version Control	Source code management, branching, merging
GitHub	Repository Hosting	Remote repository, pull requests, code reviews
Apache Kafka	Distributed Systems	Event streaming concepts and enterprise messaging study
Elasticsearch	Search & Analytics	Search optimization and distributed indexing study
Python / OpenCV	AI & Vision	Emerging technology exploration
HTML / CSS	Frontend	Structured layout and responsive styling

Plagiarism Report

The plagiarism report must be signed by the faculty member. Only the first page of the report should be included in the final project submission.

[Attach plagiarism report here]